Distributed live stacking: home-network test checklist

The six remaining manual tasks (T208, T310, T417, T507, T606, T705) and how to test everything locally

2026-10-22

How to use this checklist

Every code task in `TASKS.DISTRIBUTED_STACKING.md` is merged. What is left needs real machines on your home network: the Mac, the Windows PC, N.I.N.A. and your own frames. This checklist runs those six tasks in an order where each one builds on the last.

- **Order.** Part 0 is the one-off setup, Part 1 the automated checks, Part 2 brings the system up. Then T417, T208, T310, T507, T606 and T705. Part 4 tears it down.
- **Names used below.** `<mac>` is the Mac's local host name (`scutil --get LocalHostName`), so the Mac is `<mac>.local` on the LAN. `SF` is the Starfront checkout (`~/Develop/EKEServices/Astro/starfront`) and `RSC` the Rapid Stacker Collab checkout (`~/Develop/EKEServices/Astro/Rust/rapid-stacker-collab`).
- **Three machines (2026-10-24).** `deploy/home-lab/README.md` does Part 0's network set-up and Part 2 with Terraform on your network: the Linux machine (`omarchy.lan`) runs the server and the hub, and the Mac, the Linux machine and the Windows PC each run a worker, with the catalogue on the NAS. In that layout read `<mac>.local` below as `omarchy.lan` wherever it names the server or the hub; the Windows steps are unchanged.
- **When a task passes,** tick its box in `SF/TASKS.DISTRIBUTED_STACKING.md` and push, or tell Claude, which ticks it and refreshes the progress page.
- **When a step fails,** keep the logs (`~/starfront-local/logs/` , the hub's and workers' logs) and note the step. Nothing here touches the cloud: the local Starfront server uses a stand-in Discord, and anyone on the network can sign in as anyone. Run it on your home network only.

Write the versions here before you start:

Item	Version or commit
Starfront `main`	
Rapid Stacker Collab `master`	
N.I.N.A. plugin	
`collab-hub --version` / `collab-worker --version`	

Part 0: one-off setup

0.1 Code and binaries on the Mac

- [ ] Pull both repositories: `git -C SF pull` and `git -C RSC pull`.
- [ ] Install Starfront's Python requirements into the Python you run it with (on this Mac, Anaconda's `python3` lacked `python-multipart`, without which the hub's results are refused and no preview ever appears):

  ```
  cd SF && python3 -m pip install -r requirements.txt
  ```

  `local_night.py` and `server/run_local.py` now refuse to start without it and print this command.
- [ ] Build the three programs (needs your rapid-stacker access; `local_night.py` also does this unless given `--no-build`):

  ```
  cd RSC/worker
  cargo build --release -p collab-hub -p collab-worker -p collab-uploader
  ```
- [ ] **Do not run the local-stack tests in your main RSC checkout.** It holds a built Lambda tree from the last deploy, so the local Node stack refuses to start there. Run them in a separate worktree, or run `backend/scripts/clean.sh` only when no deploy is pending.

0.2 Windows binaries

- [ ] On GitHub, open Rapid Stacker Collab, Actions, **worker-release**, Run workflow on `master`. It is manual and publishes nothing.
- [ ] Download the `worker-release-windows-x64` artifact. Unzip `collab-worker-<version>-windows-x64.zip` to `C:\collab-worker\`.
- [ ] Download the `worker-release-macos-arm64` artifact too and keep it for 0.5. You need all three artifacts only if you cut a release.

0.3 Network and firewall

- [ ] The Mac and the PC are on the same LAN, and the PC's network is set to **Private**.
- [ ] On the PC: `ping <mac>.local` answers.
- [ ] On the Mac: allow incoming connections for `python3` (port 8800) and `collab-hub` (port 8790) when macOS asks. If you use the macOS firewall in stealth mode, add both by hand.

0.4 The XPSD catalogue share

- [ ] On the Mac: System Settings, General, Sharing, File Sharing. Add `~/Astronomy/XPSD`, and give your Windows account **Read only**.
- [ ] On the PC: `dir \\<mac>\XPSD\gdr3-*.xpsd` lists the 16 band files. Map it as a drive if you like; a logon-task worker sees your mapped drives.

0.5 The release key (only T507 needs it)

Until the real key replaces the placeholders, every desktop install and every worker self-update is refused **by design**. The steps are in `RSC/docs/runbooks/releasing-workers.md`, sections 2 and 3.

- [ ] Generate the key with OpenSSL 3 (Homebrew's, not macOS's LibreSSL). Keep `release-private.pem` offline, never in a repository.
- [ ] Paste the printed public key into `SF/astrocontrol/volunteer.py` (`MANIFEST_PUBLIC_KEY`) and into `RSC/worker/crates/worker/src/self_update.rs` (`RELEASE_PUBLIC_KEY`). Commit both, rebuild the worker and re-run worker-release.
- [ ] Build and sign both manifests from the three artifacts (`build-manifest.sh`, `sign-manifest.sh`, `--verify`), with `--base-url http://<mac>.local:8800/download/<product>` for this local test.
- [ ] Copy the archives, `latest.json` and `latest.json.sig` into `SF/server/downloads/collab-worker/` (and `collab-hub/`). Those files are git-ignored; the local server serves them from there.

Part 1: automated checks first

Run these on the Mac before any manual task. If one fails, fix it before going on. Times are from the last runs.

- [ ] **The whole hub with real workers and the real uploader** (about 3 minutes; one worker, two workers, a killed worker):

  ```
  cd RSC && bash scripts/hub-e2e.sh --only all
  ```

- [ ] **Server, hub, worker and the pages together**, with synthetic frames, on spare ports and a scratch folder:

  ```
  cd SF && python3 tools/local_night.py --synthetic --exit-when-stacked \
      --data ~/starfront-ln-check --port 18800 --hub-port 18790
  ```

  Expected: it ends with `live preview published` and exits 0. Delete `~/starfront-ln-check` afterwards.

- [ ] **Starfront's checks.** Every `tools/check_*.py` must pass except the three already failing before this work (`check_coordinator`, `check_night_rollover`, `check_reference`):

  ```
  cd SF && for f in tools/check_*.py; do python3 "$f" >/dev/null 2>&1 && echo "PASS $f" || echo "FAIL $f"; done
  ```

  The ones this work added or changed are `check_server`, `check_stack_relay`, `check_hub`, `check_volunteer`, `check_csp`, `check_notify`, `check_controls` and `check_local_server`.

- [ ] **The contract**: `cd RSC && node scripts/check-contract.mjs` reports `no mismatches` (needs `npm ci` in `backend/` once).

- [ ] **The plugin's tests** (on the PC, or the Mac with .NET 8): `cd SF/nina && dotnet test tests/Starfront.Collab.Tests`.

- [ ] **CI on GitHub.** After your next push to RSC, open the `hub-e2e` job in worker-ci. It has never run on a GitHub runner.

Part 2: bring the home network up

Every manual task uses this. Leave it running in its own terminal.

- [ ] On the Mac, start everything. For real frames, give your target's centre and a field a little larger than your frames (Elephant's Trunk shown):

  ```
  cd SF && python3 tools/local_night.py --region 323.7,57.5,2,2 \
      --discord-webhook https://discord.com/api/webhooks/<id>/<token>
  ```

  The webhook is optional; it is needed only for T705. Use a test channel's webhook (channel settings, Integrations, Webhooks).

- [ ] Copy the banner it prints. It shows the server and hub addresses, the project page, the stacking project id (`prj_…`), a **Windows enrol code** (single use, 1 hour), the uploader commands, and the `ServerUrl` for N.I.N.A.

- [ ] Note the file locations: everything is under `~/starfront-local/` . The hub's config is `hub/collab-hub.toml` , its data root `hub/data/` , the telescope grant `telescope-grant` (valid 24 hours) and the logs `logs/` .

- [ ] Make an admin token for the status checks below, and keep it in this terminal:

  ```
  export COLLAB_HUB_ADMIN_TOKEN=$(RSC/worker/target/release/collab-hub token --admin --config ~/starfront-local/hub/collab-hub.toml)
  RSC/worker/target/release/collab-hub status --url http://127.0.0.1:8790
  ```

  Expected: a status document with one healthy worker (`mac-worker`), the catalogue `ready` and a recent `starfront_last_sync` .

Part 3: the six tasks

T417: server, hub and pages on the home network

`local_night.py` has already started the server with `--lan` , started the hub, registered it and bound a collaboration.

- [ ] On the PC, open `http://<mac>.local:8800/stack/p/<slug>` from the banner. *Expected:* the project page renders; it is empty until frames arrive.

- [ ] Upload a few real lights from the Mac through the server's front door, using the three uploader lines from the banner (`projects` first: it refreshes the uploader's list of joined projects):

```
COLLAB_TOKEN=$(cat ~/starfront-local/telescope-grant) collab-uploader \
    --api-base http://<mac>.local:8800/stack/api --config-dir ~/starfront-local/uploader/real projects
COLLAB_TOKEN=... collab-uploader ... rig confirm --from /path/to/one/light.fits --yes
COLLAB_TOKEN=... collab-uploader ... upload /path/to/folder --project prj_… --include-calibration --now
```

- [ ] *Expected:* the hub pulls the frames, stacks them and pushes back. Within a few minutes the page on the PC shows the frames, the per-filter previews and the contributor "Local scope".

- [ ] Reload `/stack/dashboard` on the PC after signing in through `/files/signin` (any name). *Expected:* "My projects" lists the collaboration with the hub's status line.

- [ ] Tick **T417**.

T208: two real workers on the home network drain a hub

- [ ] **A Mac service worker**, to test launchd. `local_night.py`'s own worker runs in the foreground, so enrol a second one in its own folder with a fresh code:

```
H=RSC/worker/target/release
CODE=$($H/collab-hub enrol-code --config ~/starfront-local/hub/collab-hub.toml --note "Mac service")
$H/collab-worker enrol http://<mac>.local:8790 --code $CODE --worker-id mac-service --dir ~/collab-worker-svc
$H/collab-worker service install --dir ~/collab-worker-svc
$H/collab-worker service status --dir ~/collab-worker-svc
```

- [ ] **The Windows worker**, in a normal Command Prompt (not an administrator one), with the code from the banner:

```
cd C:\collab-worker
collab-worker.exe enrol http://<mac>.local:8790 --code XXXX-XXXX --worker-id pc-windows
collab-worker.exe service install
collab-worker.exe service status
```

  Expected: `collab-worker.toml` in `%APPDATA%\collab-worker\` has `xpsd_dir = '\\<mac>\XPSD'`, and Task Scheduler shows a logon task.

- [ ] On the Mac, `collab-hub status --url http://127.0.0.1:8790`. *Expected:* `mac-worker`, `mac-service` and `pc-windows` are all healthy with `catalogue: ready`.

- [ ] **Drain.** Upload a folder of real frames (the T417 commands, or the T310 data). *Expected:* `GET /api/admin/fleet` (or the status) shows jobs on more than one worker, and every queue goes back to 0. The Windows worker's log (`service status`) shows prepares.

- [ ] **Catalogue missing.** On the PC, disconnect the share (`net use \\<mac>\XPSD /delete`, or disconnect the mapped drive). *Expected:* within 5 minutes the status shows `pc-windows` with the catalogue `missing`. Reconnect it and see `ready` again.

- [ ] **Busy file.** On the Mac, add `busy_file = "/Users/<you>/collab-worker-svc/busy"` to `~/collab-worker-svc/collab-worker.toml` and run `collab-worker service restart --dir ~/collab-worker-svc`. Then `touch ~/collab-worker-svc/busy`. *Expected:* `mac-service` shows `busy` with the reason `imaging` and takes no new work. Delete the file and it resumes. This works only with the idle policy `when_idle`, which is the default for a hub worker.

- [ ] Tick **T208**.

T310: real data through the hub

T310 was written as a "standalone hub". Projects now only come from Starfront, so it runs on the Part 2 setup. The hub is the same program; only the project comes from the local server.

- [ ] Use the Elephant's Trunk subset: upload it with the T417 uploader commands. The `--region` you gave `local_night.py` must contain the frames' centre, or plate solving fails.
- [ ] *Expected:* the Windows worker prepares some of the frames, reading the catalogue through the Mac's share (its log shows prepares; the hub's status shows no catalogue errors). Lights end `accepted` with registration figures; the live stack grows.
- [ ] Force a gold run instead of waiting for the hourly scheduler:

  ```
  RSC/worker/target/release/collab-hub gold-now --data ~/starfront-local/hub/data
  ```

- [ ] Render the hub's previews with the website smoke test (needs Chrome):

  ```
  cd RSC/website && node scripts/browser-smoke.mjs --sim ~/starfront-local/hub/data/objects/public
  ```

  Expected: every page passes, and the previews look like your target.

- [ ] Tick **T310**.

T507: a volunteer installs from the desktop

Needs 0.5 done. Without the real key the install is refused, and that refusal is the correct behaviour.

A known gap: the hub's mirror of the worker manifest (plan 4.6) was never built. The desktop tries the hub's mirror, gets nothing, and falls back to the server's `/download/collab-worker/latest.json`. The install still works; it is fetched from the Mac's Starfront server, not the hub.

- [ ] On the PC, first remove the hand-installed worker from T208: `collab-worker.exe service uninstall`.
- [ ] Build the installer on the PC (it must include `cryptography` and the new pages): `cd SF && packaging\build.ps1`, then install `Starfront-Setup-<version>.exe`.
- [ ] In the desktop, point it at the local server (`collab.serverUrl = http://<mac>.local:8800`), sign in through the stand-in Discord, open the Collab tab, then **Volunteer this machine**.
- [ ] Enter the hub address `http://<mac>.local:8790` and a fresh enrol code (`collab-hub enrol-code` on the Mac). Leave **Pause while N.I.N.A. is open** on. Press **Install**.
- [ ] *Expected:* the log says the manifest verified, the archive's SHA-256 matched, the worker enrolled and the logon task was installed. The Mac's `collab-hub status` lists the new worker.
- [ ] Open N.I.N.A. *Expected:* within a heartbeat the worker shows `busy` with the reason "N.I.N.A. is running" and takes no work. Close N.I.N.A. and it resumes.

- [ ] Optional: on the Mac, open the desktop's **Live stacking hub** section and check that the fleet table shows the same workers.
- [ ] Tick **T507**.

T606: the hub checklist in N.I.N.A.

- [ ] On the PC, build and install the plugin (`SF/nina/MANUAL_CHECKLIST.md`, section A). Set `ServerUrl = http://<mac>.local:8800` in the plugin's settings file.
- [ ] Run **section F, "Hub projects"** of `SF/nina/MANUAL_CHECKLIST.md`, box by box. It covers: the hub binding in the listing; no Collab sign-in needed; the join; frames through the server while the hub is down; frames over the LAN short-cut while it is up; verdicts in the dock; falling back when the hub stops; removing the telescope; a refused grant and its recovery; a cloud and a hub collaboration on the same night; and the SQLite preload.
- [ ] Tick **T606**.

T705: the full night

- [ ] Start Part 2 with `--discord-webhook` set (or restart `local_night.py` with it), then point N.I.N.A. on the PC at it.
- [ ] Image a real target that is inside `--region`. **Over the LAN short-cut:** *Expected:* each light shows `accepted` in the Starfront dock and in the live preview on `/stack/p/<slug>` within about a minute.
- [ ] **Through the server relay:** block the hub's LAN port from the PC only, in an administrator PowerShell:

  ```
  New-NetFirewallRule -DisplayName "Block collab hub" -Direction Outbound -Protocol TCP -RemotePort 8790 -Action Block
  ```

  Expected: the plugin logs that it now uploads through the Starfront server; lights still reach `accepted` within about a minute, relayed by the server. Remove the rule afterwards: `Remove-NetFirewallRule -DisplayName "Block collab hub"`.
- [ ] **Gold.** At the end of the night, or with `collab-hub gold-now --data ~/starfront-local/hub/data`. *Expected:* a gold master is published, and the page shows it.
- [ ] **Discord.** *Expected:* the test channel receives "member joined" when the telescope first joined, and "gold published" with hours and contributors.
- [ ] Tick **T705**.

Part 4: tear down

- [ ] On the Mac: `collab-worker service uninstall --dir ~/collab-worker-svc`, then `rm -rf ~/collab-worker-svc`.

- [ ] On the PC: `collab-worker.exe service uninstall` (and the desktop's **Uninstall** under Volunteer, if used). Remove the firewall rule if it is still there.
- [ ] Stop `local_night.py` with Ctrl-C. It stops the server, the hub and its worker.
- [ ] Optional: keep or delete `~/starfront-local/`. It holds the hub's database and the masters; `collab-hub backup` keeps a copy.
- [ ] Optional: turn off File Sharing of the XPSD folder.

Known gaps found while writing this

None of these block the six tasks. They are worth a decision.

Gap	Effect	Where
Release key is a placeholder	every desktop install and self-update is refused until 0.5	`volunteer.py`, `self_update.rs`
The hub's worker-manifest mirror was never built (no task covered it)	the desktop installs from the Starfront server instead of the hub	plan 4.6; `releasing-workers.md` §4
No desktop field for a collaboration's Discord webhook	set it with `local_night.py --discord-webhook`, or the server's API	`server/app.py` `discordWebhookUrl`
The Starfront health route has no `stackContract`	nothing reports the contract the server's front door speaks	plan 7.3
The `hub-e2e` CI job has not yet run on GitHub	a first run may need fixing	`RSC/.github/workflows/worker-ci.yml`